ECE 526L Spring 2025 Lab 5

Design of 8-bit Up Counter using Reset Synchronizer (AASD)



Muhammad Umar
CSUN ID: 204533244

Instructor: Prof. Orod Haghighiara

I hereby attest that this lab report is entirely my own work. I have not copied either code or text from anyone, nor have I allowed or will I allow anyone to copy my work.

Name (printed): **Muhammad Umar**

Name (signed):  Date: 03/15/2025

INTRODUCTION

In this lab, an 8-bit reloadable up-counter with an Asynchronous Assert, Synchronous Deassert (**AASD**) reset synchronizer is designed and implemented. The counter includes control signals for enabling counting, loading a specific value, and maintaining its state based on synchronous conditions.

The asynchronous reset ensures immediate assertion when activated, while the synchronizer guarantees a stable deassertion on the clock edge to prevent **metastability** issues. The counter increments on each clock cycle when enabled, unless the load signal is asserted, in which case it loads the given data value. The design follows a modular approach, with separate modules for the counter and reset synchronizer, instantiated at the top level.

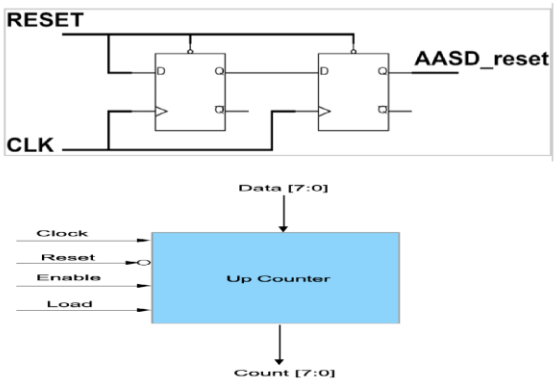


Figure 5.1 AASD and Up Counter Schematic

A testbench is developed to verify the counter's functionality under various conditions as required, including asynchronous reset behavior, enable control, parallel loading, counter rollover, and reset priority. This implementation demonstrates fundamental concepts of digital design, including clocked sequential logic, metastability handling, and modular design techniques.

METHODOLOGY AND TEST PLAN

1. Counter Module:

We designed an 8-bit up counter with several key features to control its operation. It includes single-bit control inputs: a **Clock** for synchronization, an asynchronous active-low **Reset** to instantly clear the counter to zero, an **Enable** signal to start or stop counting, and a **Load** signal to update the counter with a specific value. The counter also has an 8-bit Data input for loading a predefined value and an 8-bit Count output to display the current counter value.

The Reset is asynchronous and active-low, meaning it resets the counter immediately, regardless of the clock. The Enable and Load signals are synchronous, taking effect only on the rising edge of the clock. When **Enable** is high, the counter increments on each clock cycle; when **low**, it holds the current value. When Load is asserted, the counter updates to the value provided at the Data input on the next clock edge. This design ensures precise control and flexibility for various counting applications.

2. Reset Synchronizer Module:

This module ensures metastability protection for the **reset** signal. When the **async_rst_n** signal is low, the reset is asserted **asynchronously**, which ensures an immediate response to reset conditions. However, when async_rst_n deasserted, the module employs a two stage flip-flop chain to synchronize the reset deassertion

with the clock. This two-stage synchronization prevents metastability by allowing sufficient time for the reset signal to stabilize before it propagates through the system. By combining asynchronous assertion and synchronized deassertion, the module guarantees reliable and glitch-free reset behavior, even in the presence of clock domain crossings or timing uncertainties.

3. Top-Level Module:

The **counter** and **reset_synchronizer** were instantiated in a top-level design, with an intermediate wire **sync_rst_n** connecting the synchronized reset signal to the counter.

4. Testbench:

A testbench was written to verify the behavior of the design under different conditions mentioned below using a 20 ns clock period.

a.Clock Generation
b. Asynchronous Reset Behavior
c. Basic Counting Functionality
d. Parallel Load Operation
e. Counter Rollover
f. Reset Overrides Load and Enable
g. Enable Functionality

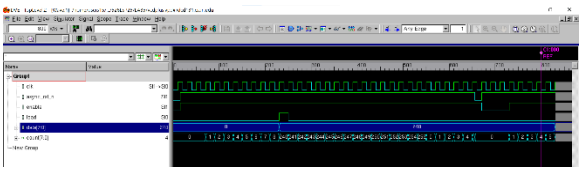


Figure 5.2 Waveform of Top Level Counter

a. Test Plan

The test plan ensures that all functional aspects of the design are verified. The following test scenarios were executed:

a. Clock Generation

- A 20 ns clock signal was generated using an always block.

b. Asynchronous Reset Behavior

- The reset was asserted (async_rst_n = 0) at the start, and the counter was expected to be reset to zero immediately.
- The reset was deasserted (async_rst_n = 1) before the next rising clock edge, and the counter started operating normally.

c. Basic Counting Functionality

- The Enable signal was asserted, allowing the counter to increment on each clock cycle.
- The counter incremented correctly until reaching a count of 8.

d. Parallel Load Operation

- The Load signal was asserted at count 8, and the Data input was set to **240**.
- The counter loaded the new value (240) on the next positive clock edge.

e. Counter Rollover

- The counter was allowed to increment from 240.
- After reaching the maximum **255**, it rolled over to 0 and continued counting up.

f. Reset Overrides Load and Enable

- While the counter was counting, async_rst_n was asserted.
- The counter was expected to immediately reset to 0, regardless of Enable or Load values.

g. Enable Functionality

- The Enable signal was deasserted, and the counter held its current value.
- When Enable was reasserted, counting resumed from the last value.

5. Extra Credit Questions

a) If the reset was **synchronous**, it would only take effect on the **rising clock edge**, potentially causing timing issues in metastable conditions. It will behave like other signals enable and load as they are also synchronous.

b) To modify the design to roll over at 245, an additional comparison condition **(if count == 245 then count <= 0)** would be added to the counter logic.

ANALYSIS OF RESULTS

The **8-bit up counter** design was successfully tested and verified for functionality and reliability. A 20 ns clock signal provided stable timing, while the asynchronous active-low reset (async_rst_n) ensured immediate initialization. The counter incremented correctly with the Enable signal, loaded values via the Load signal, and demonstrated proper rollover from 255 to 0. The reset overrode Enable and Load, and the Enable signal effectively paused and resumed counting. The design met all requirements, offering reliability and flexibility for integration into larger systems. Future enhancements could include customizable rollover values, such as rolling over at 245.

CONCLUSION

The **8-bit up counter** design was successfully implemented and verified through comprehensive testing, demonstrating its functionality and reliability. The counter performed as expected in all test scenarios, including clock generation, asynchronous reset, counting, parallel loading, rollover, and enable control. The asynchronous reset ensured immediate and reliable initialization, while the synchronous Enable and Load signals provided precise control over the counter's operation. The test results confirmed that the design meets the specified requirements and behaves correctly under various conditions. For future enhancements, the counter could be modified to support customizable rollover values, such as rolling over at 245 instead of 255, by adding a simple comparison condition. Overall, the design is robust, flexible, and well-suited for integration into larger digital systems. All the design codes, testbench and log files are in appendices.

APPENDIX

Appendix A: reset_synchronizer.v

```
/*************************************************************************
***                                                                    ***
*** EE 526 L Experiment #5            Muhammad Umar, Spring, 2025      ***
***                                                                    ***
*** Experiment #5 Design 8-bit Reloadable up counter with AASD Reset    ***
*** Synchronizer.                                                      ***
***                                                                    ***
*** Filename: reset_synchronizer.v  Created by Muhammad Umar, 03/10/2025 ***
***                                                                    ***
*** This module implements a logic circuit of Asynchronous Assert      ***
*** Synchronous Deassert (AASD), When reset is 0 then it will reset the  ***
*** circuit at the same time, when reset is 1, the circuit will get the  ***
*** reset signal stable on positive edge of clock cycle to avoid        ***
*** metastability condition                                            ***
*************************************************************************/

module reset_synchronizer (
   input wire clk,
   input wire async_rst_n, // Asynchronous active low reset
   output wire sync_rst_n  // Synchronized active low reset
);

   reg rst_ff1, rst_ff2; // Two flip-flops used for synchronization

   always @(posedge clk or negedge async_rst_n) begin
      if (!async_rst_n) begin
         rst_ff1 <= 1'b0;
         rst_ff2 <= 1'b0;
      end else begin
         rst_ff1 <= 1'b1;
         rst_ff2 <= rst_ff1;
      end
   end

   assign sync_rst_n = rst_ff2; // Synchronized reset output

endmodule
```

Appendix B: up_counter.v

```
/*************************************************************************
***                                                                    ***
*** EE 526 L Experiment #5            Muhammad Umar, Spring, 2025      ***
***                                                                    ***
*** Experiment #5 Design 8bit Reloadable up counter with AASD Reset    ***
*** Synchronizer.                                                      ***
***                                                                    ***
*** Filename: up_counter.v        Created by Muhammad Umar, 03/10/2025 ***
***                                                                    ***
*** This module implements a logic circuit of up_counter with load and ***
*** and enable signal.                                                 ***
*************************************************************************/
module up_counter (
   input wire clk,
   input wire rst_n,       // Asynchronous active-low reset
   input wire enable,      // Synchronous active high enable
   input wire load,        // Synchronous active high load
   input wire [7:0] data,
   output reg [7:0] count  // count output
);

   // Asynchronous reset and rest is synchronous because of AASD module
   always @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
         count <= 8'b0; // Reset the counter to 0
      end else if (enable) begin
         if (load) begin
            count <= data; // Load the data into the counter
         end else begin
            count <= count + 1; // Incrementing the counter when enable is active and load is 0.
         end
      end
   end

endmodule
```

Appendix C: top_level.v

```
/*************************************************************************
***                                                                    ***
*** EE 526 L Experiment #5            Muhammad Umar, Spring, 2025      ***
***                                                                    ***
*** Experiment #5 Design 8-bit Reloadable up counter with AASD Reset    ***
*** Synchronizer.                                                      ***
***                                                                    ***
*** Filename: top_level.v         Created by Muhammad Umar, 03/10/2025  ***
***                                                                    ***
*** This module implements a logic circuit of Asynchronous Assert      ***
*** Synchronous Deassert (AASD) and up-counter module by instantiating  ***
*** both the modules, where an intermediate wire sync_rst_n will get the ***
*** synchroized reset signal from reset_synchronizer module and then it  ***
*** will be given to up_counter module                                  ***
*************************************************************************/

module top_level (
   input wire clk,
   input wire async_rst_n,   // Asynchronous active low reset
   input wire enable,        // Synchronous active high enable
   input wire load,          // Synchronous active high load
   input wire [7:0] data,
   output wire [7:0] count   // count output
);

   wire sync_rst_n; // Synchronized reset signal from AASD module

   // Instantiate the reset synchronizer
   reset_synchronizer sync_reset (
      .clk(clk),
      .async_rst_n(async_rst_n),
      .sync_rst_n(sync_rst_n)
   );

   // Instantiate the counter
   up_counter counter (
      .clk(clk),
      .rst_n(sync_rst_n),
      .enable(enable),
      .load(load),
      .data(data),
      .count(count)
```

```
    );

endmodule
```

Appendix D: top_level_tb.v

```
/*************************************************************************
***                                                                    ***
*** EE 526 L Experiment #5            Muhammad Umar, Spring, 2025      ***
***                                                                    ***
*** Experiment #5 Design 8-bit Reloadable up counter with AASD Reset    ***
*** Synchronizer.                                                      ***
***                                                                    ***
*** Filename: top_level_tb.v      Created by Muhammad Umar, 03/10/2025  ***
***                                                                    ***
*** This module implements a testbench of logic circuit of Top Level   ***
*** in which we have identified all the cases                          ***

*************************************************************************/
module top_level_tb;
   reg clk;
   reg async_rst_n;
   reg enable;
   reg load;
   reg [7:0] data;
   wire [7:0] count;

   // (a) Instantiating the top-level module
   top_level uut (
      .clk(clk),
      .async_rst_n(async_rst_n),
      .enable(enable),
      .load(load),
      .data(data),
      .count(count)
   );

   // (b) Clock generation of 20 ns period.
   initial begin
      clk = 0;
      forever #10 clk = ~clk;
   end

   // Test sequence
   initial begin
         $vcdpluson;
```

```
        $display("\n*****************************************************************
**********");
    $display("Initializing signals and asserting asynchronous reset\n");
        $display("*****************************************************************
*********\n");
    async_rst_n = 1'b0;
    enable = 1'b0;
    load = 1'b0;
    data = 8'b0;

        $display("*****************************************************************
*********");
    #15 $display("Deasserting reset before positive edge to check async behavior\n");
        $display("*****************************************************************
*********\n");

    async_rst_n = 1'b1;

    // (d) Enable the counter and let it count to 8
        $display("*****************************************************************
*********");
    $display("Enabling counter and waiting for it to count to 8\n");
        $display("*****************************************************************
*********\n");
    enable = 1'b1;
    repeat (11) @(posedge clk);

    // (e) Load 240 into the counter when it reaches 8
        $display("*****************************************************************
*********");
    $display("Loading 240 into counter\n");
        $display("*****************************************************************
*********\n");
    data = 8'd240;
    load = 1'b1;
    @(posedge clk) load = 1'b0;

    // Let the counter count until it rolls over
        $display("*****************************************************************
*********");
```

```
      $display("Letting counter count until it rolls over\n");
         $display("*****************************************************************
*********\n");
      repeat (20) @(posedge clk);

      // (f) Demonstrate reset override
         $display("*****************************************************************
*********");
      #5 $display("Asserting reset between edges to check immediate effect\n");
         $display("*****************************************************************
*********\n");
      async_rst_n = 1'b0;
      #15 $display("Deasserting reset before next positive edge\n");
      async_rst_n = 1'b1;

      // (g) Demonstrate enable functionality
         $display("*****************************************************************
*********");
      $display("Disabling counter to check enable functionality\n");
         $display("*****************************************************************
*********\n");
      enable = 1'b0;
      repeat (4) @(posedge clk);
      $display("Enabling counter again\n");
      enable = 1'b1;

      repeat (5) @(posedge clk);
         $display("*****************************************************************
*********");
      $display("                              Simulation Ended");
         $display("*****************************************************************
*********\n");
      $finish;
   end

   // Monitor signals
   initial begin
      $monitor("\n Time = %0t | Reset = %b | Enable = %b | Load = %b | Data = %d | Count =
%d\n",
               $time, async_rst_n, enable, load, data, count);
```

```
    end
endmodule
```

Appendix E: top_level.log

```
Command: /home/users15/mu832615/526LSP25/LAB5/./simv -l top_level.log
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version U-2023.03-SP1_Full64; Runtime version U-2023.03-SP1_Full64;
Mar 16 17:19 2025
VCD+ Writer U-2023.03-SP1_Full64 Copyright (c) 1991-2023 by Synopsys Inc.

*************************************************************************
Initializing signals and asserting asynchronous reset

*************************************************************************

*************************************************************************

 Time = 0 | Reset = 0 | Enable = 0 | Load = 0 | Data =   0 | Count =   0

Deasserting reset before positive edge to check async behavior

*************************************************************************

*************************************************************************
Enabling counter and waiting for it to count to 8

*************************************************************************


 Time = 15 | Reset = 1 | Enable = 1 | Load = 0 | Data =   0 | Count =   0


 Time = 70 | Reset = 1 | Enable = 1 | Load = 0 | Data =   0 | Count =   1


 Time = 90 | Reset = 1 | Enable = 1 | Load = 0 | Data =   0 | Count =   2


 Time = 110 | Reset = 1 | Enable = 1 | Load = 0 | Data =   0 | Count =   3


 Time = 130 | Reset = 1 | Enable = 1 | Load = 0 | Data =   0 | Count =   4


 Time = 150 | Reset = 1 | Enable = 1 | Load = 0 | Data =   0 | Count =   5
```

```
 Time = 170 | Reset = 1 | Enable = 1 | Load = 0 | Data =   0 | Count =   6

 Time = 190 | Reset = 1 | Enable = 1 | Load = 0 | Data =   0 | Count =   7

 Time = 210 | Reset = 1 | Enable = 1 | Load = 0 | Data =   0 | Count =   8

**********************************************************************
Loading 240 into counter

**********************************************************************

 Time = 230 | Reset = 1 | Enable = 1 | Load = 1 | Data = 240 | Count = 240

**********************************************************************
Letting counter count until it rolls over

**********************************************************************

 Time = 250 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 241

 Time = 270 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 242

 Time = 290 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 243

 Time = 310 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 244

 Time = 330 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 245

 Time = 350 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 246

 Time = 370 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 247

 Time = 390 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 248

 Time = 410 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 249
```

```
 Time = 430 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 250

 Time = 450 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 251

 Time = 470 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 252

 Time = 490 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 253

 Time = 510 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 254

 Time = 530 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count = 255

 Time = 550 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   0

 Time = 570 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   1

 Time = 590 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   2

 Time = 610 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   3

 Time = 630 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   4
**********************************************************************

 Time = 650 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   5
Asserting reset between edges to check immediate effect
**********************************************************************

 Time = 655 | Reset = 0 | Enable = 1 | Load = 0 | Data = 240 | Count =   0
Deasserting reset before next positive edge
**********************************************************************
Disabling counter to check enable functionality
```

```
**************************************************************************

 Time = 670 | Reset = 1 | Enable = 0 | Load = 0 | Data = 240 | Count =   0

Enabling counter again

 Time = 730 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   1

 Time = 750 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   2

 Time = 770 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   3

 Time = 790 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   4

 Time = 810 | Reset = 1 | Enable = 1 | Load = 0 | Data = 240 | Count =   5

**************************************************************************
                  Simulation Ended
**************************************************************************

$finish called from file "top_level_tb.v", line 85.
$finish at simulation time                  830
          V C S   S i m u l a t i o n   R e p o r t
Time: 830
CPU Time:      0.180 seconds;       Data structure size:   0.0Mb
Sun Mar 16 17:19:40 2025
```